

# Reporte Técnico: Arquitectura y Patrones del Proyecto

**Proyecto:** Druvle Stock | **Fecha:** 11 de mayo de 2026 | **Tipo:** Estado actual

## 1. Resumen ejecutivo

El proyecto se encuentra implementado como un **monolito modular en Laravel**, con base **MVC** y una separación adicional por capas de aplicación (servicios, repositorios, eventos/listeners y jobs). Esta estructura es adecuada para mantener evolución funcional sin necesidad de migrar a microservicios en la etapa actual.

**Conclusión:** Arquitectura sólida para su escala actual, con diseño extensible y desacoplado en flujos críticos (notificaciones y eventos de negocio).

## 2. Tipo de arquitectura

### 2.1 Monolito modular

- Una sola aplicación desplegable, con módulos organizados por responsabilidad.
- Módulos funcionales en controladores backend: ventas, productos, reportes, notificaciones, usuarios, tenants.

### 2.2 Base MVC + capas

- **Presentación:** rutas + controladores + vistas Blade.
- **Negocio:** servicios y lógica coordinada por controladores.
- **Persistencia:** Eloquent y repositorios para consultas especializadas.
- **Procesamiento reactivo:** eventos, listeners y colas.

## 3. Patrones de diseño aplicados

### 3.1 MVC

- Controladores orquestan casos de uso y retornan vistas/JSON.
- Modelos gestionan entidades y relaciones del dominio.

### 3.2 Service Layer

- La lógica de notificaciones está centralizada en `NotificationService`.
- Permite reutilización, menor duplicación y control consistente de reglas.

### 3.3 Repository Pattern

- Consultas de reportes y ventas encapsuladas en repositorios dedicados.
- Controladores no necesitan conocer detalles de consultas complejas.

### 3.4 Event-Driven / Observer

- Eventos de negocio disparados desde procesos clave (ventas y devoluciones).
- Listeners reaccionan para tareas secundarias desacopladas (ej. notificaciones).

### 3.5 Async Processing con colas

- Listeners críticos implementan `ShouldQueue`.
- Evita bloquear flujos principales por operaciones secundarias.

### 3.6 Dependency Injection (IoC)

- Inyección por constructor en controladores y listeners.
- Mejora testabilidad y reduce acoplamiento directo.

### 3.7 Active Record (Eloquent)

- Modelos representan tablas, relaciones y parte de comportamiento.
- Patrón natural del stack Laravel usado de forma consistente.

### 3.8 Multitenancy transversal (Scope + Trait + Middleware)

- Scope global para filtrar automáticamente por tenant.
- Trait reutilizable aplicado a múltiples modelos con `tenant_id`.
- Middleware para resolver tenant activo y compartirlo en contexto.

#### 4. Flujo funcional principal (alto nivel)

- La petición entra por rutas web y middleware.
- El controlador valida y coordina el caso de uso.
- Se persiste/consulta mediante modelos o repositorios.
- Si aplica, se dispara evento de dominio.
- Listeners procesan acciones secundarias, algunos en cola.

#### 5. Fortalezas actuales

- Separación de responsabilidades superior al MVC básico.
- Desacoplamiento por eventos en procesos relevantes.
- Soporte multi-tenant integrado y reutilizable.
- Estructura mantenible para crecimiento incremental.

#### 6. Observaciones y oportunidades

- Hay controladores con lógica extensa que podrían migrarse a servicios de dominio.
- Parte de consultas se repite entre controladores y servicios.
- La escalabilidad futura depende más de operación (Redis, workers, observabilidad) que de un cambio arquitectónico radical.

#### 7. Conclusión final

El proyecto está bien diseñado para su etapa: **monolito modular con patrones correctos y enfoque práctico**. La arquitectura permite evolucionar sin deuda crítica estructural inmediata.

Fin del reporte.